

Redes Neuronales

Aprendizaje Maquinal

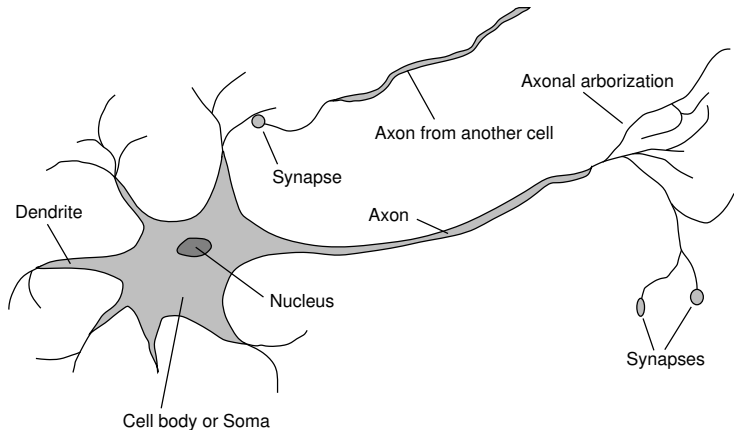
René Mac Kinney Romero

UAM - Iztapalapa

- Cerebros
- Redes Neuronales
- Perceptrones
- Perceptrones Multicapa
- Aplicaciones de redes neuronales

Cerebros

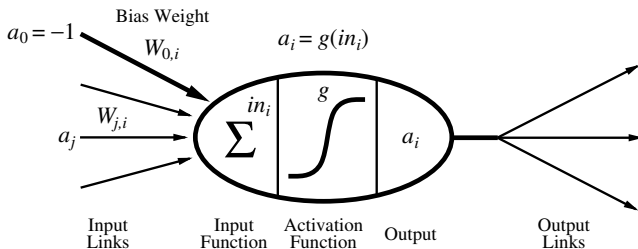
10^{11} neuronas de > 20 tipos, 10^{14} sinapsis, 1ms–10ms tiempo de respuesta
Las señales son ruidosos “trenes de puntas” de potenciales eléctricos



“unidad” McCulloch–Pitts

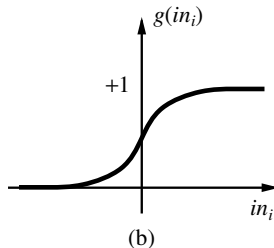
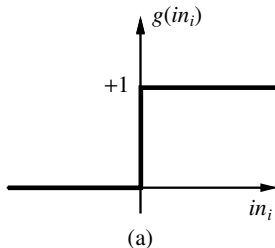
La salida es una función lineal “aplastada” de las entradas:

$$a_i \leftarrow g(in_i) = g \left(\sum_j w_{j,i} a_j \right)$$



Una burda sobre-simplificación de una neurona real, pero su propósito es desarrollar comprensión de que una red de unidades simples pueden hacer

Funciones de activación



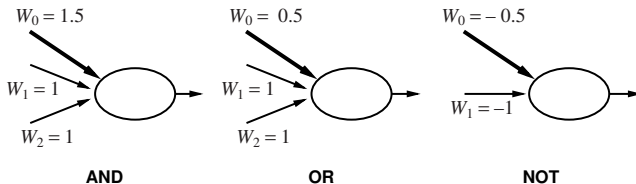
(a) es una **función escalón** o una **función de umbral**

(b) es una función **sigmoidea** $P(x) = 1/(1 + e^{-x})$ (muy usada porque $\frac{dP}{dx} = P(1 - P)$)

Cambiando el peso del sesgo¹ $W_{0,i}$ mueve el lugar del umbral

¹ *bias*

Implementando funciones lógicas



McCulloch and Pitts: toda función Booleana puede ser implementada

Redes Feed-forward:

- perceptrones de capa única
- perceptrones multicapa

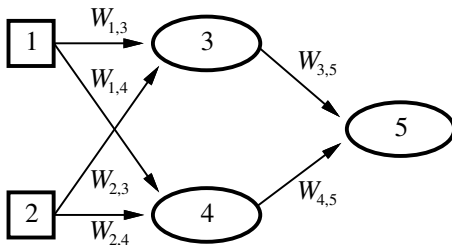
Las redes Feed-forward implementan funciones, no tienen estado interno

Redes Recurrentes:

- Redes de Hopfield tienen pesos simétricos ($W_{i,j} = W_{j,i}$)
 $g(x) = \text{sign}(x)$, $a_i = \pm 1$; **memoria asociativa holográfica**
- Maquinas de Boltzmann usan funciones estocásticas de activación,
 $\approx$ MCMC en redes de Bayes
- redes recurrentes neuronales tienen ciclos dirigidos con pausas
 $\implies$ tienen estado interno (como flip-flops), pueden oscilar

etc.

Ejemplo feed-forward

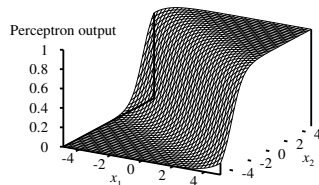
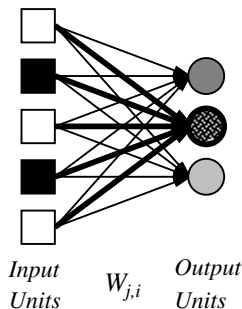


Red Feed-forward = una familia paramétrica de funciones no lineales:

$$\begin{aligned} a_5 &= g(W_{3,5} \cdot a_3 + W_{4,5} \cdot a_4) \\ &= g(W_{3,5} \cdot g(W_{1,3} \cdot a_1 + W_{2,3} \cdot a_2) + \\ &\quad W_{4,5} \cdot g(W_{1,4} \cdot a_1 + W_{2,4} \cdot a_2)) \end{aligned}$$

Ajustando los pesos cambia la función: ¡así se aprende!

Perceptrones de una capa

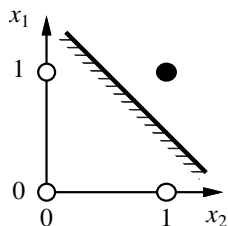


Las unidades de salida operan separadamente—no hay pesos compartidos
Ajustando los pesos se mueve el lugar, la orientación, e inclinación de
acantilado

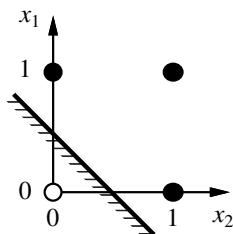
Expresividad de los perceptrones

Considere un perceptrón con $g =$ función escalón (Rosenblatt, 1957, 1960)
Puede representar AND, OR, NOT, mayoría, etc., pero no XOR
Representa un **separador lineal** en el espacio de entrada:

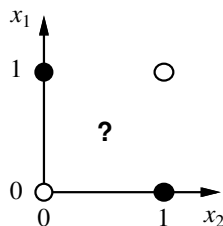
$$\sum_j W_j x_j > 0 \quad \text{or} \quad \mathbf{W} \cdot \mathbf{x} > 0$$



(a) x_1 **and** x_2



(b) x_1 **or** x_2



(c) x_1 **xor** x_2

Minsky & Papert (1969) reventaron el globo de las redes neuronales

Aprendizaje del Perceptrón

Aprende ajustando pesos para reducir el **error** en el conjunto de entrenamiento

El **error al cuadrado** para un ejemplo con entrada x y salida verdadera y es

$$E = \frac{1}{2} Err^2 \equiv \frac{1}{2} (y - h_{\mathbf{W}}(x))^2 ,$$

Realiza búsqueda del óptimo por descenso de gradiente:

$$\begin{aligned} \frac{\partial E}{\partial W_j} &= Err \times \frac{\partial Err}{\partial W_j} = Err \times \frac{\partial}{\partial W_j} \left(y - g\left(\sum_{j=0}^n W_j x_j\right) \right) \\ &= -Err \times g'(in) \times x_j \end{aligned}$$

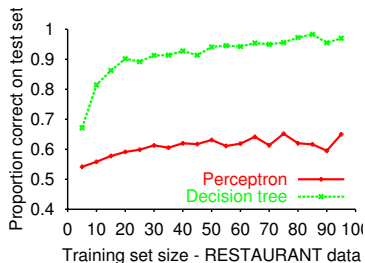
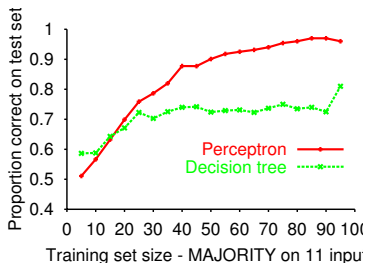
Regla simple de actualización de pesos:

$$W_j \leftarrow W_j + \alpha \times Err \times g'(in) \times x_j$$

E.g., +ve error $\implies$ incrementa salida de la red $\implies$ incrementa pesos en las entradas +ve, decrementa en entradas -ve

Aprendizaje del Perceptrón cont.

La regla de aprendizaje del perceptrón converge a una función consistente **para cualquier conjunto linealmente separable**

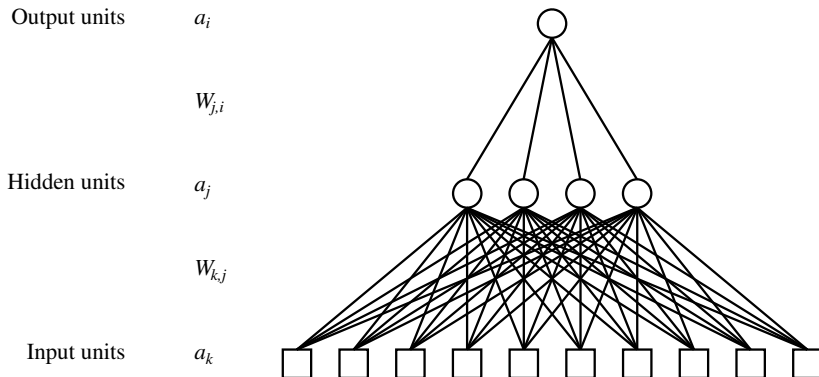


El perceptrón aprende la función mayoría fácilmente, DTL² es inútil
DTL aprende la función del restaurante fácilmente, el perceptrón no puede representarlo

²Decision Tree Learning Aprendizaje de árboles de decisión

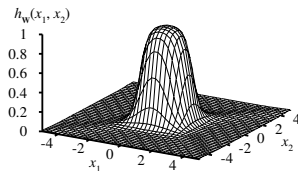
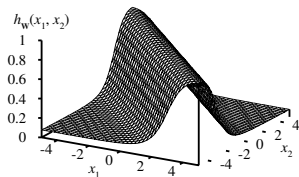
Perceptrones Multicapa

Los niveles están usualmente completamente conectados;
número de **unidades** típicamente se eligen a mano



Expresividad de perceptrones multicapa

Todas las funciones continuas c/2 capas, todas las funciones c/3 capas



Combina dos funciones de umbrales con caras opuestas para hacer una cordillera

Combina dos cordilleras perpendiculares para crear un tope

Agregue topes de varios tamaños y posiciones para embonar en cualquier superficie

La demostración requiere exponencialmente de muchas unidades ocultas

Aprendizaje por retro-propagación

Capa de salida: igual que para el perceptrón de una capa,

$$W_{j,i} \leftarrow W_{j,i} + \alpha \times a_j \times \Delta_i$$

donde $\Delta_i = Err_i \times g'(in_i)$

Capa oculta: **retro-propaga** el error de la capa de salida:

$$\Delta_j = g'(in_j) \sum_i W_{j,i} \Delta_i .$$

Regla de actualización de pesos para capa oculta:

$$W_{k,j} \leftarrow W_{k,j} + \alpha \times a_k \times \Delta_j .$$

(La mayoría de los neurocientíficos niegan que la retro-propagación ocurra en el cerebro)

Derivación de la retro-propagación

El error al cuadrado de un solo ejemplo se define como

$$E = \frac{1}{2} \sum_i (y_i - a_i)^2 ,$$

donde la suma es sobre los nodos de la capa de salida.

$$\begin{aligned} \frac{\partial E}{\partial W_{j,i}} &= -(y_i - a_i) \frac{\partial a_i}{\partial W_{j,i}} = -(y_i - a_i) \frac{\partial g(in_i)}{\partial W_{j,i}} \\ &= -(y_i - a_i) g'(in_i) \frac{\partial in_i}{\partial W_{j,i}} = -(y_i - a_i) g'(in_i) \frac{\partial}{\partial W_{j,i}} \left(\sum_j W_{j,i} a_j \right) \\ &= -(y_i - a_i) g'(in_i) a_j = -a_j \Delta_i \end{aligned}$$

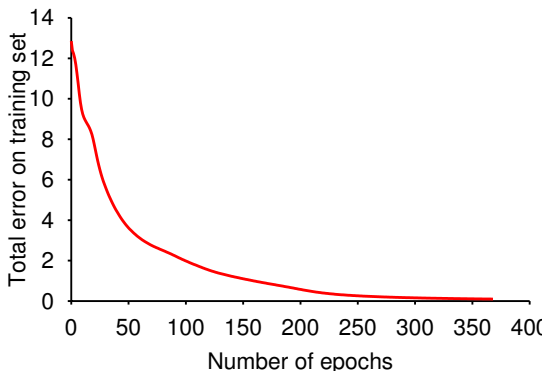
Derivación de la retro-propagación contd.

$$\begin{aligned}\frac{\partial E}{\partial W_{k,j}} &= -\sum_i (y_i - a_i) \frac{\partial a_i}{\partial W_{k,j}} = -\sum_i (y_i - a_i) \frac{\partial g(in_i)}{\partial W_{k,j}} \\&= -\sum_i (y_i - a_i) g'(in_i) \frac{\partial in_i}{\partial W_{k,j}} = -\sum_i \Delta_i \frac{\partial}{\partial W_{k,j}} \left(\sum_j W_{j,i} a_j \right) \\&= -\sum_i \Delta_i W_{j,i} \frac{\partial a_j}{\partial W_{k,j}} = -\sum_i \Delta_i W_{j,i} \frac{\partial g(in_j)}{\partial W_{k,j}} \\&= -\sum_i \Delta_i W_{j,i} g'(in_j) \frac{\partial in_j}{\partial W_{k,j}} \\&= -\sum_i \Delta_i W_{j,i} g'(in_j) \frac{\partial}{\partial W_{k,j}} \left(\sum_k W_{k,j} a_k \right) \\&= -\sum_i \Delta_i W_{j,i} g'(in_j) a_k = -a_k \Delta_j\end{aligned}$$

Aprendizaje por retro-propagación contd.

En cada **época**, aplica actualización con la suma de gradientes de todos los ejemplos

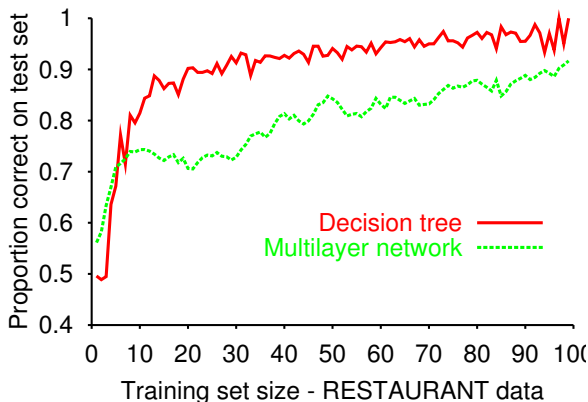
Curva de entrenamiento para ejemplo de 100 restaurantes: encuentra pesos exactos



Problemas típicos: convergencia lenta, mínimos locales

Aprendizaje por retro-propagación contd.

Curva de aprendizaje para MLP con cuatro unidades ocultas:



Los MLPs son muy buenos para tareas complejas de reconocimiento de patrones,
pero las hipótesis resultantes pueden no ser fácilmente entendibles

Reconocimiento de dígitos escritos a mano



3-vecinos-mas-cercanos = 2.4 % error

MLP con 400–300–10 unidades = 1.6 % error

LeNet: MLP 768–192–30–10 unidades = 0.9 % error

El mejor actualmente (kernel machines, algoritmos de visión) $\approx$ 0.6 % error

La mayoría de los cerebros tienen muchas neuronas; cada neurona $\approx$ unidad de umbral-lineal ($\hat{?}$)

Los perceptrones (redes de una capa) no son suficientemente expresivos

Las redes multi-capas son suficientemente expresivas; pueden entrenarse por descenso de gradiente, i.e., retro-propagación de error

Muchas aplicaciones: habla, manejar, OCR, detección de fraudes, etc.

Ingeniería, modelos cognitivos, y modelos de sistemas neuronales

los subcampos han divergido grandemente